Programmation quadratique séquentielle (SQP)

Analyse, implémentation et application à la régression logistique régularisée par contrainte

Imene AIT ABDELLAH
Cours d'optimisation – Projet

November 7, 2026

Abstract

Ce rapport étudie la méthode SQP (*Sequential Quadratic Programming*) pour l'optimisation non linéaire sous contraintes. Nous présentons la théorie (conditions KKT, sous-problème quadratique, globalisation, convergence, complexité), puis une implémentation *from scratch* en Python (sous-solveur QP à ensemble actif, BFGS avec amortissement de Powell, fonction de mérite ℓ_1, correction de second ordre). La méthode est validée sur des problèmes à solution connue et comparée à `SLSQP` et `trust-constr` de SciPy. Elle est ensuite appliquée à la régression logistique dont la régularisation est écrite comme une *contrainte explicite* $\|w\|_2^2 \leq t$ (ridge) ou $\|w\|_1 \leq t$ (lasso) sur le jeu de données Breast Cancer Wisconsin. Nous montrons l'équivalence avec la forme pénalisée de `scikit-learn`, le chemin de régularisation, et analysons la vitesse et la robustesse, ainsi que les limites de la méthode.

Contents

1 Introduction et motivation **3**

2 Théorie **3**
2.1 Problème général et conditions KKT 3
2.2 Le sous-problème quadratique 4
2.3 Choix de B_k 4
2.4 Globalisation : fonction de mérite ℓ_1 et recherche linéaire 5
2.5 Contraintes linéarisées incompatibles : mode élastique 5
2.6 Conditions de convergence 5
2.7 Complexité 6
2.8 Cas d'usage typiques 6
2.9 Comparaison avec d'autres méthodes 6

3 Implémentation **6**
3.1 Sous-solveur QP à ensemble actif 6
3.2 Boucle SQP 7
3.3 Vérification des dérivées 7

4 Validation **7**
4.1 Problèmes à solution connue 7
4.2 Cas limites 8

5 Application : régression logistique contrainte **8**
5.1 Données et modèle 8
5.2 Équivalence contrainte / pénalisé 8
5.3 Chemin de régularisation, biais–variance et choix de t 9
5.4 Version lasso : parcimonie 9

6 Analyse des performances **9**
6.1 Courbes de convergence 9
6.2 Vitesse comparée 10
6.3 Robustesse 10

7 Discussion et limites **11**
7.1 Ce qui marche 11
7.2 Ce qui ne marche pas, ou moins bien 11
7.3 Difficultés rencontrées 11

8 Conclusion **12**

1 Introduction et motivation

Le problème d'apprentissage. On dispose de données $(x_i, y_i)_{i=1}^n$, avec $x_i \in \mathbb{R}^p$ et $y_i \in \{0, 1\}$, et d'un modèle qui prédit la probabilité $p_i = \sigma(b + w^\top x_i)$, où $\sigma(z) = 1/(1 + e^{-z})$. On cherche (b, w) qui minimise la log-loss moyenne

$$J(b, w) = -\frac{1}{n} \sum_{i=1}^{n} \Big[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \Big]. \tag{1}$$

Avec $\tilde{x}_i = (1, x_i)$, le gradient et le Hessien s'écrivent

$$\nabla J = \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i)\, \tilde{x}_i, \qquad \nabla^2 J = \frac{1}{n} \sum_{i=1}^{n} p_i(1 - p_i)\, \tilde{x}_i \tilde{x}_i^\top. \tag{2}$$

Le Hessien est semi-défini positif : J est convexe. Le problème est donc facile en théorie, mais il présente deux pathologies :

- **Données séparables** : on peut toujours diminuer la perte en augmentant $\|w\|$ dans la direction qui sépare les classes. La perte tend vers 0 sans jamais l'atteindre, donc aucun minimiseur fini n'existe.
- **Peu de données, beaucoup de variables, variables corrélées** : le Hessien est mal conditionné, les poids deviennent très grands et instables, et le modèle surapprend (variance élevée).

Dans les deux cas, rien n'empêche $\|w\|$ de croître. Limiter la taille de w est précisément le rôle de la régularisation.

Beaucoup de problèmes d'apprentissage s'écrivent comme la minimisation d'une perte sous contraintes. La régularisation en est l'exemple le plus courant : au lieu de pénaliser $\|w\|^2$ avec un coefficient λ, on peut imposer directement un *budget* sur les poids,

$$\min_{b,w} \; J(b, w) \quad \text{s.c.} \quad \|w\|_2^2 \le t. \tag{3}$$

Les deux écritures sont équivalentes (section 5.2), mais la forme contrainte a un intérêt pédagogique : elle fait apparaître un multiplicateur de Lagrange interprétable et justifie l'emploi d'une méthode d'optimisation sous contraintes comme SQP.

La méthode SQP est une référence pour les problèmes de taille petite à moyenne avec contraintes non linéaires. Son principe est simple : *à chaque itération, remplacer le problème par un problème quadratique avec contraintes linéaires, le résoudre, et avancer.* Elle peut être vue comme la méthode de Newton appliquée aux conditions d'optimalité de Karush–Kuhn–Tucker (KKT).

Objectifs. (i) Exposer la théorie de SQP ; (ii) implémenter la méthode, y compris le sous-solveur QP ; (iii) la valider ; (iv) l'appliquer à la régression logistique contrainte ; (v) évaluer erreur, vitesse et robustesse, et en identifier honnêtement les limites.

Organisation. Section 2 : théorie. Section 3 : implémentation. Section 4 : validation. Section 5 : application. Section 6 : performances. Section 7 : discussion. Section 8 : conclusion.

2 Théorie

2.1 Problème général et conditions KKT

On considère

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{s.c.} \quad g(x) \le 0, \quad h(x) = 0, \tag{4}$$

avec $f : \mathbb{R}^n \to \mathbb{R}$, $g : \mathbb{R}^n \to \mathbb{R}^{m_i}$, $h : \mathbb{R}^n \to \mathbb{R}^{m_e}$ de classe C^2. Le lagrangien est

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \lambda^\top g(x) + \mu^\top h(x). \tag{5}$$

Sous une condition de qualification (LICQ : les gradients des contraintes actives sont linéairement indépendants), tout minimiseur local $x^\star$ admet des multiplicateurs $(\lambda^\star, \mu^\star)$ tels que

$$\nabla_x \mathcal{L}(x^\star, \lambda^\star, \mu^\star) = \nabla f(x^\star) + J_g(x^\star)^\top \lambda^\star + J_h(x^\star)^\top \mu^\star = 0 \qquad \text{(stationnarité)} \tag{6}$$

$$g(x^\star) \le 0, \quad h(x^\star) = 0 \qquad \text{(faisabilité primale)} \tag{7}$$

$$\lambda^\star \ge 0 \qquad \text{(faisabilité duale)} \tag{8}$$

$$\lambda_i^\star \, g_i(x^\star) = 0 \quad \forall i \qquad \text{(complémentarité)} \tag{9}$$

Ce sont les conditions KKT. Dans le code, elles sont vérifiées numériquement à la solution (`checks.kkt_residuals`) et servent de critère d'arrêt.

2.2 Le sous-problème quadratique

À l'itéré x_k, on approche l'objectif par un modèle quadratique et les contraintes par leur linéarisation. La direction d_k est solution de

$$\min_d \ \tfrac{1}{2} d^\top B_k d + \nabla f(x_k)^\top d \quad \text{s.c.} \quad g(x_k) + J_g(x_k) d \le 0, \quad h(x_k) + J_h(x_k) d = 0, \tag{10}$$

où $B_k \approx \nabla^2_{xx} \mathcal{L}(x_k, \lambda_k, \mu_k)$. Le QP (10) fournit aussi les nouveaux multiplicateurs $(\lambda_{k+1}, \mu_{k+1})$. On pose ensuite $x_{k+1} = x_k + \alpha_k d_k$ où α_k est choisi par recherche linéaire (section 2.4).

Lien avec la méthode de Newton. Pour des contraintes d'égalité seules, les conditions KKT s'écrivent $F(x, \mu) = (\nabla_x \mathcal{L}, \ h) = 0$. Une itération de Newton sur F donne le système linéaire

$$\begin{pmatrix} \nabla^2_{xx} \mathcal{L}_k & J_h^\top \\ J_h & 0 \end{pmatrix} \begin{pmatrix} d_k \\ \mu_{k+1} \end{pmatrix} = - \begin{pmatrix} \nabla f(x_k) \\ h(x_k) \end{pmatrix}, \tag{11}$$

qui est exactement le système KKT du QP (10) avec $B_k = \nabla^2_{xx} \mathcal{L}_k$. SQP est donc Newton sur KKT ; le QP permet en plus de traiter correctement les inégalités (c'est lui qui décide quelles contraintes sont actives).

2.3 Choix de B_k

Hessien exact. $B_k = \nabla^2 f + \sum_i \lambda_i \nabla^2 g_i + \sum_j \mu_j \nabla^2 h_j$. Il donne la convergence quadratique locale mais peut être indéfini loin de la solution. Dans notre implémentation, il est rendu défini positif (troncature des valeurs propres) et régularisé par un terme $\tau_k I$:

$$B_k = \mathrm{PD}(\nabla^2_{xx} \mathcal{L}_k) + \tau_k I, \qquad \tau_k = \min\left(1, \|\nabla_x \mathcal{L}_k\|_\infty\right), \tag{12}$$

où $\tau_k \to 0$ près de la solution (la convergence rapide est préservée) mais évite des pas gigantesques quand la courbure est presque nulle loin de $x^\star$.

BFGS avec amortissement de Powell. Sans dérivées secondes, on met à jour B_k par BFGS. Avec $s_k = x_{k+1} - x_k$ et $y_k = \nabla_x \mathcal{L}(x_{k+1}, \lambda_{k+1}, \mu_{k+1}) - \nabla_x \mathcal{L}(x_k, \lambda_{k+1}, \mu_{k+1})$, la condition de courbure $s_k^\top y_k > 0$ n'est pas garantie sous contraintes. L'amortissement de Powell remplace y_k par $r_k = \theta_k y_k + (1 - \theta_k) B_k s_k$ avec

$$\theta_k = \begin{cases} 1 & \text{si } s_k^\top y_k \ge 0{,}2 \, s_k^\top B_k s_k, \\ \dfrac{0{,}8 \, s_k^\top B_k s_k}{s_k^\top B_k s_k - s_k^\top y_k} & \text{sinon,} \end{cases} \qquad B_{k+1} = B_k - \frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k} + \frac{r_k r_k^\top}{s_k^\top r_k}, \tag{13}$$

ce qui garantit que B_{k+1} reste symétrique définie positive.

2.4 Globalisation : fonction de mérite ℓ_1 et recherche linéaire

Le pas complet $\alpha = 1$ peut diverger loin de la solution. On mesure le progrès avec la fonction de mérite exacte

$$\phi_\pi(x) = f(x) + \pi\, v(x), \qquad v(x) = \sum_i [g_i(x)]_+ + \sum_j |h_j(x)|, \tag{14}$$

qui combine objectif et violation. Notons $v_{\text{lin}}(d) = \sum_i [g_i + (J_g d)_i]_+ + \sum_j |h_j + (J_h d)_j|$ la violation du modèle linéarisé. La dérivée directionnelle de ϕ_π en x_k le long de d_k est majorée par

$$D_k = \nabla f(x_k)^\top d_k - \pi\big(v(x_k) - v_{\text{lin}}(d_k)\big). \tag{15}$$

Le paramètre de pénalité est mis à jour pour que $D_k < 0$:

$$\pi \leftarrow \max\Big(\pi,\ 1{,}1\,\|(\lambda_{k+1}, \mu_{k+1})\|_\infty,\ \frac{\nabla f^\top d_k + \tfrac{1}{2} d_k^\top B_k d_k}{\tfrac{1}{2}\,(v(x_k) - v_{\text{lin}}(d_k))}\Big). \tag{16}$$

La recherche linéaire d'Armijo (backtracking $\alpha \leftarrow \alpha/2$) cherche le premier α tel que

$$\phi_\pi(x_k + \alpha d_k) \le \phi_\pi(x_k) + \eta\,\alpha\, D_k, \qquad \eta = 10^{-4}. \tag{17}$$

Effet Maratos et correction de second ordre. Près de la solution, la linéarisation des contraintes commet une erreur d'ordre $\|d\|^2$; cette violation de second ordre peut faire rejeter le pas complet par ϕ_π alors qu'il est excellent (effet Maratos), ce qui détruit la convergence rapide. On corrige par une *correction de second ordre* (SOC) : si le pas complet est refusé, on résout à nouveau le QP avec des constantes de contraintes évaluées au point d'essai, $g(x_k + d_k) - J_g d_k$ et $h(x_k + d_k) - J_h d_k$, et on teste le pas obtenu. Cette correction a été indispensable dans notre code (section 7).

2.5 Contraintes linéarisées incompatibles : mode élastique

Le QP (10) peut être infaisable (point initial éloigné, contraintes non linéaires). On le relâche avec des variables d'écart $p, q, s \ge 0$ pénalisées en ℓ_1 :

$$\min_{d,p,q,s}\ \tfrac{1}{2} d^\top B d + c^\top d + \rho\, \mathbf{1}^\top (p + q + s) + \tfrac{\varepsilon}{2}(\|p\|^2 + \|q\|^2 + \|s\|^2) \tag{18}$$

$$\text{s.c. } J_h d + h = p - q, \qquad J_g d + g \le s, \qquad p, q, s \ge 0. \tag{19}$$

Ce QP est *toujours faisable* ($d = 0$ avec des écarts adaptés), donc résoluble par un ensemble actif primal démarré en un point faisable connu. Si le QP original est faisable et ρ plus grand que les multiplicateurs, les écarts sont nuls à la solution (pénalité exacte). Nous prenons $\rho = 10^3$, $\varepsilon = 10^{-3}$ et multiplions ρ par 10 (jusqu'à 4 fois) si un écart subsiste.

2.6 Conditions de convergence

Locale. Si $x^\star$ vérifie LICQ, les conditions suffisantes du second ordre et la complémentarité stricte, et si (x_0, λ_0, μ_0) est assez proche de $(x^\star, \lambda^\star, \mu^\star)$, alors SQP avec Hessien exact converge *quadratiquement* ; avec une approximation quasi-Newton qui vérifie la condition de Dennis–Moré, la convergence est *superlinéaire*. De plus, sous ces hypothèses, le QP identifie l'ensemble des contraintes actives en un nombre fini d'itérations (Nocedal & Wright, chap. 18).

Globale. Avec la fonction de mérite ϕ_π, π assez grand et B_k uniformément définie positive, les itérés ont des points d'accumulation qui sont des points KKT (ou des points stationnaires de la violation) (Nocedal & Wright, §18.3 et suivants).

2.7 Complexité

Soit n le nombre de variables et m le nombre de contraintes. Une itération de SQP coûte :

- l'évaluation de f, ∇f, g, h et des jacobiennes (et du Hessien si exact : $O(n^2)$ stockage) ;
- la résolution du QP : chaque itération interne de l'ensemble actif factorise un système KKT de taille $n + m_{\text{actif}}$, soit $O\big((n+m)^3\big)$ en dense ; le nombre d'itérations internes croît en général avec le nombre de contraintes ;
- la mise à jour BFGS : $O(n^2)$.

La **mémoire** est $O(n^2)$ (matrice B_k). Le **nombre d'itérations** SQP est typiquement faible (de l'ordre de 5 à 50) mais chacune est chère : la méthode est adaptée aux problèmes de petite à moyenne taille avec contraintes non linéaires, pas à l'apprentissage profond à grand n.

2.8 Cas d'usage typiques

Apprentissage régularisé par contrainte (ce projet), apprentissage sous contraintes d'équité (contraintes non linéaires sur les prédictions), calibration de modèles, contrôle optimal et trajectoires (peu de variables, dynamique non linéaire), conception d'ingénierie, finance (optimisation de portefeuille).

2.9 Comparaison avec d'autres méthodes

Méthode	**Contraintes**	**Vitesse locale**	**Coût / itér.**	**Limites**
Plus forte pente (+ projection / pénalité)	Projection (si ensemble simple) ou pénalité	Linéaire, lente si mal conditionné	$O(n)$	Choix du pas, mauvais conditionnement
BFGS (non contraint)	Aucune native ; via pénalité	Superlinéaire	$O(n^2)$	Ne gère pas les contraintes seul
Méthode de pénalité	Pénalisation $\mu\,[g]_+^2$	Dépend du solveur interne	Variable	Mal conditionnement quand $\mu \to \infty$, contraintes satisfaites seulement à la limite
SQP	Égalités et inégalités non linéaires	Superlinéaire / quadratique	$O((n+m)^3)$	Mémoire $O(n^2)$, coût du QP, linéarisations infaisables

Table 1: Comparaison qualitative de catégories de méthodes.

3 Implémentation

Tout le code est en Python (NumPy). Le sous-solveur QP et la fonction de mérite sont implémentés par nos soins ; SciPy n'est utilisé que pour la *comparaison*.

3.1 Sous-solveur QP à ensemble actif

Nous résolvons le QP convexe $\min \frac{1}{2} z^\top H z + g^\top z$ s.c. $Ez = e$, $Cz \geq r$ par la méthode d'ensemble actif primal (Nocedal & Wright, Alg. 16.3), démarrée en un point faisable connu :

1. résoudre le QP à égalités sur l'ensemble de travail $\mathcal{W}$ (système KKT linéaire) ;

2. si la direction est nulle : si tous les multiplicateurs sont ≥ 0, arrêt ; sinon retirer de $\mathcal{W}$ la contrainte de multiplicateur le plus négatif ;
3. sinon, avancer jusqu'à la première contrainte bloquante, qui est ajoutée à $\mathcal{W}$ (en vérifiant l'indépendance linéaire).

Grâce au mode élastique, le point de départ $d = 0$ (avec écarts adaptés) est toujours faisable. Les multiplicateurs finaux sont recalculés par moindres carrés sur les contraintes actives seules (système mieux conditionné que le système KKT complet).

3.2 Boucle SQP

1. Évaluer $f, \nabla f, g, J_g, h, J_h$; $B_k \leftarrow$ BFGS ou Hessien exact régularisé.
2. Résoudre le QP (10) (élastique) $\Rightarrow d_k, \lambda_{k+1}, \mu_{k+1}$.
3. **Test d'arrêt** : $\|\nabla_x \mathcal{L}\|_\infty \leq$ tol, violation $\leq$ tol, résidu de complémentarité $\leq$ tol (ou pas $\|d\|_\infty \leq 10^{-10}(1 + \|x\|_\infty)$ avec contraintes satisfaites, utile en mauvais conditionnement) ; tol $= 10^{-8}$ par défaut.
4. Mettre à jour π ; calculer D_k ; recherche linéaire d'Armijo (avec SOC si le pas complet est refusé).
5. $x_{k+1} = x_k + \alpha_k d_k$; mise à jour de B (Powell) ; journaliser f, violation, $\|d\|$, α, π, temps.

3.3 Vérification des dérivées

Tous les gradients, jacobiennes et Hessiens analytiques sont comparés à des différences finies centrées. L'erreur relative maximale est de l'ordre de 10^{-10} sur tous les problèmes (tableau 2), pour un seuil de 10^{-5}.

Problème	∇f	J_g ou J_h	Hessien
Rosenbrock–disque	$4 \cdot 10^{-11}$	$8 \cdot 10^{-11}$	–
HS6	$1 \cdot 10^{-10}$	$7 \cdot 10^{-11}$	–
HS7	$7 \cdot 10^{-11}$	$2 \cdot 10^{-10}$	–
Logistique ℓ_2	$1 \cdot 10^{-10}$	$3 \cdot 10^{-11}$	$3 \cdot 10^{-10}$
Logistique ℓ_1	$4 \cdot 10^{-11}$	$7 \cdot 10^{-11}$	$1 \cdot 10^{-10}$

Table 2: Erreur relative maximale entre dérivées analytiques et différences finies.

4 Validation

4.1 Problèmes à solution connue

Trois problèmes sont utilisés :

- **Rosenbrock sous disque** : $\min 100(x_2 - x_1^2)^2 + (1 - x_1)^2$ s.c. $x_1^2 + x_2^2 \leq 1$, $x^\star \approx (0{,}7864,\ 0{,}6177)$;
- **HS6** (égalité) : $\min (1 - x_1)^2$ s.c. $10(x_2 - x_1^2) = 0$, $x^\star = (1, 1)$;
- **HS7** (égalité) : $\min \ln(1 + x_1^2) - x_2$ s.c. $(1 + x_1^2)^2 + x_2^2 - 4 = 0$, $x^\star = (0, \sqrt{3})$.

Tableau 3 compare notre SQP (BFGS et Hessien exact) avec `SLSQP` et `trust-constr`. Toutes les méthodes trouvent la même solution à la tolérance près ; les résidus KKT de notre solveur sont de l'ordre de 10^{-9} à 10^{-12} (stationnarité) et $\leq 5 \cdot 10^{-9}$ (faisabilité).

	SQP-BFGS	SQP-exact	SLSQP	trust-constr
Rosenbrock–disque	10	16	30	76
HS6	10	10	9	12
HS7	10	8	10	9

Table 3: Nombre d'itérations. Dans tous les cas $\|x - x^\star\| \leq 7 \cdot 10^{-6}$.

4.2 Cas limites

- **Point initial infaisable** ($x_0 = (3, 3)$, hors du disque) : convergence vers la bonne solution en 30 itérations, grâce au mode élastique.
- **Contrainte inactive** (rayon^2 = 10) : la solution est le minimiseur libre $(1, 1)$ et le multiplicateur est nul ($\lambda = 0$).
- **Contrainte active** (rayon$^2 = 1$) : $g(x^\star) = 0$ et $\lambda^\star \approx 0{,}121 > 0$.
- **Mauvais conditionnement** (quadratique de conditionnement 10^6 avec une égalité) : 3 itérations dans les deux variantes, erreur $\approx 10^{-12}$.

5 Application : régression logistique contrainte

5.1 Données et modèle

Nous utilisons **Breast Cancer Wisconsin (Diagnostic)** (`sklearn`) : 569 échantillons, 30 variables numériques, cible binaire (212 malins, 357 bénins). Séparation 70 %/30 % stratifiée, standardisation sur l'ensemble d'entraînement ; l'ordonnée à l'origine b n'est pas contrainte. La perte est la log-loss moyenne

$$J(b, w) = \frac{1}{n} \sum_{i=1}^{n} \Big[\ln\left(1 + e^{z_i}\right) - y_i z_i \Big], \quad z_i = b + x_i^\top w, \tag{20}$$

de gradient $\frac{1}{n} \sum_i (\sigma(z_i) - y_i)(1, x_i)$ et de Hessien $\frac{1}{n} X_a^\top \mathrm{diag}(\sigma(z)(1 - \sigma(z))) X_a$ avec $X_a = [\mathbf{1}\ X]$. La perte n'est pas quadratique : SQP a donc un vrai travail de non-linéarité à faire.

5.2 Équivalence contrainte / pénalisé

Le lagrangien de (3) est $J + \lambda(\|w\|^2 - t)$. `scikit-learn` minimise $\frac{1}{2}\|w\|^2 + C \sum_i L_i$; en divisant par Cn on obtient $J + \frac{1}{2Cn}\|w\|^2$. Les deux problèmes ont donc la même solution lorsque

$$\boxed{C = \frac{1}{2n\lambda^\star}} \tag{21}$$

où $\lambda^\star$ est le multiplicateur fourni par notre SQP. Tableau 4 confirme l'équivalence : les poids coïncident avec `LogisticRegression` à mieux que 10^{-6}.

t	iter.	$\|w\|^2$	$\lambda^\star$	$C = 1/(2n\lambda^\star)$	$\max \lvert w_{\mathrm{SQP}} - w_{\mathrm{sk}} \rvert$	accuracy test
0,5	8	0,5000	0,1691	0,007	$1{,}9 \cdot 10^{-8}$	0,942
2,0	8	2,0000	0,0241	0,052	$5{,}9 \cdot 10^{-8}$	0,953
8,0	10	8,0000	0,0024	0,513	$7{,}7 \cdot 10^{-7}$	0,965

Table 4: Équivalence entre régression logistique contrainte (SQP) et pénalisée L2 (`scikit-learn`). Contrainte active : $\|w\|^2 = t$.

5.3 Chemin de régularisation, biais–variance et choix de t

Figure 1 montre, pour 18 valeurs de t (toutes ont convergé), la norme des poids, l'accuracy et la log-loss en entraînement et en test. Pour t petit, le modèle est trop contraint (fort biais) : l'erreur est élevée en train comme en test. Quand t augmente, la perte d'entraînement continue de diminuer, tandis que la log-loss de test atteint un minimum puis *remonte* : c'est le surapprentissage (faible biais, forte variance), visible aussi dans l'écart croissant entre accuracy d'entraînement et de test.

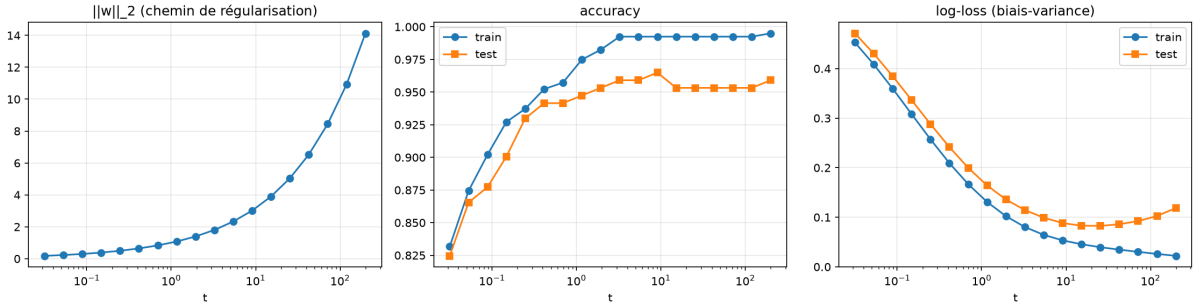


Figure 1: Chemin de régularisation (contrainte ℓ_2) : norme des poids, accuracy et log-loss en train et test en fonction de t (échelle logarithmique).

Le paramètre t est choisi par validation croisée à 5 plis (log-loss) : $t^\star \approx 9{,}1$. Sur le jeu de test, nous obtenons accuracy $= 0{,}965$, F1 $= 0{,}972$ et log-loss $= 0{,}088$, avec la matrice de confusion

$$\begin{pmatrix} 61 & 3 \\ 3 & 104 \end{pmatrix} \tag{22}$$

(lignes : classe réelle ; colonnes : classe prédite ; ordre malin, bénin).

5.4 Version lasso : parcimonie

Avec $\|w\|_1 \le t$, on pose $w = u - v$, $u, v \ge 0$ et les variables $x = (b, u, v)$. Le problème devient différentiable avec $2p + 1$ contraintes linéaires : $\mathbf{1}^\top(u + v) \le t$, $u \ge 0$, $v \ge 0$. (À l'optimum, $u_i v_i = 0$: utiliser u_i et v_i simultanément gaspillerait du budget.) Tableau 5 montre la parcimonie : plus t est petit, plus de poids sont exactement nuls.

t	iter.	$\|w\|_1$	poids non nuls (sur 30)	accuracy test
0,5	7	0,500	2	0,772
1,5	9	1,500	3	0,906
4,0	11	4,000	6	0,953

Table 5: Régression logistique avec contrainte ℓ_1 (SQP, Hessien exact).

6 Analyse des performances

6.1 Courbes de convergence

Figure 2 compare les deux variantes de SQP sur la régression logistique ($t = 1$). Avec le Hessien exact, le résidu KKT passe par $10^{-2}, 2 \cdot 10^{-3}, 5 \cdot 10^{-5}, 3 \cdot 10^{-8}, 2 \cdot 10^{-14}$: c'est une convergence **quadratique**. Avec BFGS, la décroissance est régulière en échelle logarithmique (droite) : une convergence **linéaire rapide**, en environ 37 itérations. Nous n'observons donc pas clairement

la convergence superlinéaire attendue en théorie pour BFGS, ce qui tient probablement à la précision d'approximation de B_k et à la tolérance fixée.

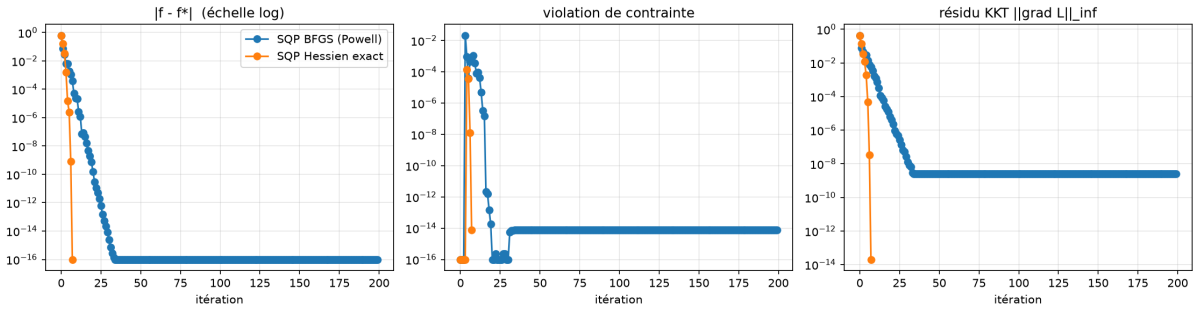


Figure 2: Convergence de SQP-BFGS et SQP-Hessien exact ($t = 1$) : $|f - f^\star|$, violation de contrainte et résidu KKT (échelle logarithmique).

6.2 Vitesse comparée

Tableau 6 compare les méthodes à $t = t^\star$ (temps médian de 5 exécutions ; dépend de la machine). Le SQP avec Hessien exact est aussi précis que possible en 10 itérations. BFGS atteint la même précision mais en 113 itérations. `SLSQP` est le plus rapide en temps (code Fortran compilé) mais moins précis ($f - f^\star \approx 10^{-6}$, violation $7 \cdot 10^{-7}$). `trust-constr` est plus lent. Une descente de gradient avec pénalité (baseline faible) ne converge pas correctement après 20 000 itérations ($f - f^\star \approx 10^{-2}$).

Méthode	**iter.**	**temps (s)**	$f - f^\star$	**violation**
SQP-BFGS (le nôtre)	113	0,095	$4 \cdot 10^{-15}$	0
SQP-Hessien exact (le nôtre)	10	0,009	0	$2 \cdot 10^{-12}$
SLSQP (SciPy)	52	0,004	$1 \cdot 10^{-6}$	$7 \cdot 10^{-7}$
trust-constr (SciPy)	89	0,131	$1 \cdot 10^{-8}$	0
Descente + pénalité	20000	0,471	$1 \cdot 10^{-2}$	0

Table 6: Comparaison des méthodes pour $t = t^\star \approx 9{,}1$.

6.3 Robustesse

Points initiaux aléatoires. Sur 20 points initiaux tirés selon $\mathcal{N}(0, 2^2)$, SQP converge vers la même solution dans 20 cas sur 20 ($49{,}4 \pm 16{,}8$ itérations avec Hessien exact).

Dimension, bruit et corrélation. Sur des données `make_classification` (1000 échantillons, 10 % de bruit sur les étiquettes, variables redondantes donc corrélées), 5 graines par dimension :

p	itérations	temps (s)
10	$7{,}2 \pm 1{,}5$	$0{,}005 \pm 0{,}001$
30	$7{,}0 \pm 0{,}6$	$0{,}007 \pm 0{,}001$
60	$7{,}2 \pm 0{,}4$	$0{,}014 \pm 0{,}002$
100	$7{,}0 \pm 0{,}0$	$0{,}018 \pm 0{,}001$

Table 7: Robustesse en dimension croissante (moyenne $\pm$ écart-type sur 5 graines).

Le nombre d'itérations est stable (autour de 7) et le temps croît modérément avec p.

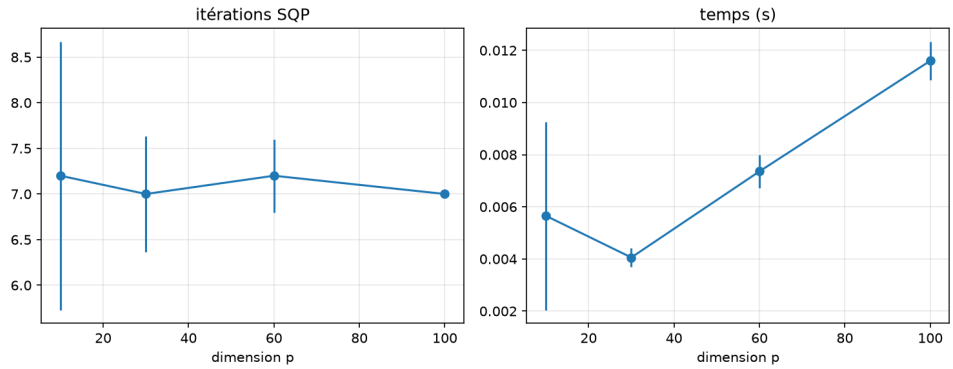


Figure 3: Itérations et temps en fonction de la dimension p (barres : écart-type sur 5 graines).

Sensibilité à la tolérance. Avec BFGS, la tolérance sur le résidu KKT fait varier le nombre d'itérations : 46 (10^{-4}), 81 (10^{-6}), 105 (10^{-8}), 126 (10^{-10}) – ce qui confirme une convergence linéaire plutôt que superlinéaire pour cette variante.

7 Discussion et limites

7.1 Ce qui marche

- Solutions identiques à `SLSQP` et `trust-constr`, conditions KKT satisfaites numériquement.
- Équivalence exacte avec la régression logistique L2 de `scikit-learn` via $C = 1/(2n\lambda^\star)$.
- Convergence quadratique avec Hessien exact (8 à 10 itérations) ; robustesse aux points initiaux infaisables et au mauvais conditionnement.
- La version lasso produit des solutions parcimonieuses ; le mode élastique permet de la traiter avec un QP à contraintes de bornes.

7.2 Ce qui ne marche pas, ou moins bien

- **BFGS** converge nettement plus lentement que le Hessien exact (113 contre 10 itérations à $t^\star$) et la convergence superlinéaire n'est pas observée ; sur ce problème, l'exactitude des dérivées secondes compte beaucoup.
- **Temps de calcul** : `SLSQP` (compilé) est plus rapide que notre implémentation Python sur ce petit problème. Notre code vise la clarté et la maîtrise, pas la vitesse.
- **Coût du QP** : l'ensemble actif dense devient coûteux quand le nombre de contraintes croît (la version lasso a $2p+1$ contraintes) et peut cycler sur des problèmes très dégénérés (un plafond d'itérations protège le code).
- **Mémoire** $O(n^2)$ et factorisations $O(n^3)$: inadapté aux problèmes de grande dimension de type apprentissage profond, où l'on préfère les méthodes de premier ordre.
- **Linéarisations infaisables** : sans le mode élastique, un point initial éloigné fait échouer le QP.

7.3 Difficultés rencontrées

Plusieurs problèmes numériques subtils ont été identifiés et corrigés pendant le développement :

1. *Pas géant avec Hessien exact* (courbure presque nulle loin de $x^\star$) $\Rightarrow$ régularisation $\tau_k I$.
2. *Multiplicateurs imprécis* en mauvais conditionnement $\Rightarrow$ recalcul par moindres carrés sur les contraintes actives.

3. *Effet Maratos masqué par les tolérances du QP* : les tolérances absolues (10^{-10}) étaient plus grandes que la correction de second ordre ($\sim 10^{-11}$), ce qui l'annulait ; resserrer ces tolérances a rétabli le pas complet.
4. *Test de descente* trop grossier pour de très petits pas $\Rightarrow$ test de signe strict ($D_k < 0$).

8 Conclusion

Nous avons présenté et implémenté une méthode SQP complète (égalités, inégalités, point initial infaisable, BFGS ou Hessien exact, mérite ℓ_1, correction de second ordre), validée contre SciPy et `scikit-learn`. Appliquée à la régression logistique avec régularisation par contrainte, elle retrouve exactement la solution pénalisée de `scikit-learn` avec le lien $C = 1/(2n\lambda^\star)$, permet de tracer le chemin de régularisation et d'obtenir des modèles parcimonieux avec la contrainte ℓ_1. Les performances sont excellentes avec le Hessien exact (convergence quadratique, environ 10 itérations) et acceptables avec BFGS. Les limites sont celles du paradigme : coût du QP, mémoire quadratique et inadaptation aux grandes dimensions.

Pistes. Hessien-vecteur et méthodes à mémoire limitée pour de plus grandes dimensions ; sous-solveur QP creux ou par points intérieurs ; contraintes d'équité pour l'apprentissage ; région de confiance à la place de la recherche linéaire.

References

[1] J. Nocedal, S. J. Wright. *Numerical Optimization*, 2e éd., Springer, 2006. Chap. 16 (programmation quadratique) et chap. 18 (SQP).

[2] M. J. D. Powell. *A fast algorithm for nonlinearly constrained optimization calculations*, 1978.

[3] W. Hock, K. Schittkowski. *Test Examples for Nonlinear Programming Codes*, Springer, 1981.

[4] W. H. Wolberg, W. N. Street, O. L. Mangasarian. *Breast Cancer Wisconsin (Diagnostic)*, UCI Machine Learning Repository.

[5] F. Pedregosa *et al. Scikit-learn: Machine Learning in Python*, JMLR 12, 2011.